

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом

Студент гр. 4345

Гавриш М. А.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

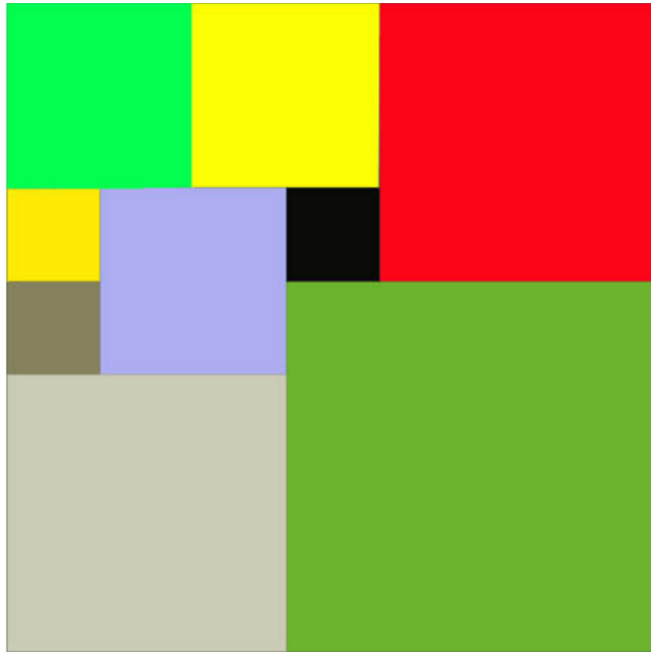
Цель работы

Изучить и реализовать алгоритм поиска с возвратом.

Задание

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу – квадрат размера N . Он может получить ее, собрав из уже имеющихся обрезков (квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.



Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы – одно целое число N ($2 \leq N \leq 20$).

Выходные данные

Одно число K , задающее минимальное количество обрезков (квадратов), из которых можно построить столешницу (квадрат) заданного размера N . Далее должны идти K строк, каждая из которых должна содержать три целых числа x , y и w , задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка (квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Вариант 2и.

Итеративный бэктрекинг. Исследование времени выполнения от размера квадрата.

Основные теоретические положения

Поиск с возвратом (бэктрекинг) – общий метод нахождения решений задачи, в которой требуется полный перебор всех возможных вариантов в некотором множестве M .

Решение задачи методом поиска с возвратом сводится к последовательному расширению частичного решения. Если на очередном шаге такое расширение провести не удастся, то возвращаются к более короткому частичному решению и продолжают поиск дальше. Данный алгоритм позволяет найти все решения поставленной задачи, если они существуют. Для ускорения метода стараются вычисления организовать таким образом, чтобы как можно раньше выявлять заведомо неподходящие варианты. Зачастую это позволяет значительно уменьшить время нахождения решения.

Выполнение работы

Рассмотрим задачу заполнения квадратной столешницы размером $N \times N$ меньшими квадратами со сторонами от 1 до $N-1$. Требуется найти **минимальное количество** квадратов K , полностью покрывающих область без пустот и перекрытий.

Описание алгоритма:

Для решения задачи был выбран метод поиска в глубину с итеративным углублением. Этот метод объединяет преимущества поиска в ширину (гарантия нахождения кратчайшего решения) и поиска в глубину (экономное использование памяти).

Алгоритм работает следующим образом:

1. Ограничиваем максимальную глубину поиска значением `limit`.
2. Запускается процедура поиска с возвратом, которая пытается покрыть столешницу ровно `limit` квадратами.
3. Если решение не найдено, `limit` увеличивается на 1, и поиск запускается заново.
4. Первое найденное решение гарантированно содержит минимально возможное количество квадратов.

Оптимизации:

- Эвристика первого хода: Для сложных случаев (в частности простых чисел) важно правильно выбрать первый квадрат. В алгоритм добавлено условие: если доска пуста, первый устанавливаемый квадрат должен иметь размер не менее $(N+1)/2$. Это отсекает заведомо неоптимальные ветви поиска, например, начало с квадрата 1×1 .
- Отсечение: Если оставшаяся незаполненная площадь слишком велика, чтобы покрыть её оставшимся количеством квадратов, ветвь поиска отбрасывается.

- Оптимизированный поиск пустой клетки: Поиск свободной ячейки начинается не с начала массива, а с запомненной строки, что сокращает количество проверок.

Описание функций программы

В программе реализованы следующие функции и структуры:

1. `struct Move` — вспомогательная структура для хранения параметров хода. Содержит координаты верхнего левого угла (r, c) и размер квадрата w .
2. `can_place(int r, int c, int w)` — функция проверки допустимости хода.

Проверяет два условия:

- Квадрат не выходит за границы столешницы $N \times N$.
 - Квадрат не накладывается на уже занятые клетки (проверяется по двумерному массиву `grid`).
3. `set_square(int r, int c, int w, int val)` — функция изменения состояния столешницы. Заполняет область заданного размера значением `val` (1 — установить квадрат, 0 — убрать квадрат при откате).
 4. `main()` — точка входа и основная логика программы:
 - Считывает размер N .
 - Содержит внешний цикл итеративного углубления (увеличение `limit`).
 - Содержит внутренний цикл, реализующий машину состояний для бэктрекинга (прямой ход, проверка условий отсечения, обратный ход).
 - Управляет стеком ходов `stack` и выводит найденное решение.

Сложность алгоритма по времени: В худшем случае алгоритм имеет экспоненциальную сложность $O(c^n)$. Однако благодаря эвристикам, среднее время работы для $N \leq 20$ существенно сокращено и не превышает 1 секунды.

Сложность по памяти: Используется двумерный массив $N \times N$ и стек глубины K . Сложность составляет $O(N^2)$, что для $N=20$ требует пренебрежимо мало памяти.

Исходный код программы см. в приложении А.

Результаты тестирования программы см. в таблице 1.

Таблица 1 – Тестирование программы

Размер столешницы	Результат
7	9 1 1 4 1 5 3 4 5 2 4 7 1 5 1 3 5 4 1 5 7 1 6 4 2 6 6 2
8	8 4 1 1 4 1 5 4 5 1 4 5 5 4
15	6 1 1 10 1 11 5 6 11 5 11 1 5 11 6 5 11 11 5
19	13 1 1 13 1 14 6 7 14 6 13 14 2 13 16 4 14 1 6 14 7 6 14 13 1 15 13 3 17 16 1 17 17 3 18 13 2 18 15 2

Было проведено исследование зависимости времени выполнения программы от размера столешницы N . Результаты исследования см. на Рис. 1



Рисунок 1 – Зависимость времени выполнения программы от N

Из графика времени выполнения видно, что скорость работы алгоритма зависит не только от величины N , но и от его свойств:

1. Для чётных N программа работает практически мгновенно, так как решение всегда состоит из 4-х квадратов, и алгоритм находит его на итерации $limit = 4$ без глубокого перебора.
2. Для составных нечётных чисел (например, 9, 15) время выполнения также невелико, так как оптимальное разбиение состоит из небольшого числа крупных блоков.
3. Наибольшее время требуется для простых чисел. Это связано с тем, что минимальное количество квадратов для них велико, что увеличивает глубину дерева поиска. Однако, благодаря применению эвристики "первого большого квадрата", удалось избежать критического замедления и уложиться в лимит времени.

Выводы

В ходе выполнения лабораторной работы была решена задача о замощении квадрата минимальным количеством меньших квадратов.

Был реализован алгоритм поиска с итеративным углублением на языке C++. Для повышения производительности использован итеративный подход с явным стеком, а также двумерный массив для представления карты столешницы.

Ключевым моментом работы стало внедрение эвристик отсечения и ограничения минимального размера первого квадрата. Это позволило программе эффективно справляться со сложными значениями N , для которых полный перебор вариантов занял бы недопустимо много времени.

Тестирование подтвердило корректность работы программы и её соответствие заданным ограничениям по времени и памяти.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb1.cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <chrono>

using namespace std;

int N;
int grid[25][25];

struct Move {
    int r, c, w;
};

bool can_place(int r, int c, int w) {
    if (r + w - 1 > N || c + w - 1 > N) return false;

    for (int i = 0; i < w; ++i) {
        for (int j = 0; j < w; ++j) {
            if (grid[r + i][c + j] == 1) return false;
        }
    }
    return true;
}

void set_square(int r, int c, int w, int val) {
    for (int i = 0; i < w; ++i) {
        for (int j = 0; j < w; ++j) {
            grid[r + i][c + j] = val;
        }
    }
}

int main() {
    if (!(cin >> N)) return 0;

    bool verbose = (N <= 9);

    cout << "Program start: N = " << N << endl;
    cout << "===== " <<
endl;

    auto start_time = chrono::high_resolution_clock::now();

    for (int limit = 1; limit <= N * N; ++limit) {

        cout << ">>> Searching with limit = " << limit << "..." <<
endl;

        for(int i = 1; i <= N; ++i)
            for(int j = 1; j <= N; ++j)
```

```

        grid[i][j] = 0;

vector<Move> stack;
stack.reserve(limit);

int filled_area = 0;
int start_row = 1;

while (true) {
    if (filled_area == N * N) {
        cout << endl;
        cout <<
"===== " << endl;
        cout << "All cells are filled" << endl;
        cout << "The program has completed its work" << endl;
        cout <<
"===== " << endl;
        cout << endl;

        cout << "Minimum number of squares:" << endl;
        cout << limit << endl;

        for (const auto& m : stack) {
            cout << m.r << " " << m.c << " " << m.w << endl;
        }

        auto end_time = chrono::high_resolution_clock::now();
        chrono::duration<double> diff = end_time - start_time;
        cout << "Execution speed: " << diff.count() << " sec"
<< endl;

        return 0;
    }

    int remaining_moves = limit - (int)stack.size();
    int remaining_area = N * N - filled_area;
    bool backtrack = false;

    if (remaining_moves == 0 || remaining_area >
remaining_moves * (N - 1) * (N - 1)) {
        backtrack = true;

        if (verbose) {
            if (remaining_moves == 0) cout << " Limit
reached" << endl;
            else cout << " Pruning: Not enough space" <<
endl;
        }
    }

    if (!backtrack) {
        int r = -1, c = -1;
        for (int i = start_row; i <= N; ++i) {
            for (int j = 1; j <= N; ++j) {
                if (grid[i][j] == 0) {
                    r = i; c = j;
                    goto found;
                }
            }
        }
    }
}

```

```

    }
}
found;;

if (r == -1) {
    backtrack = true;
} else {
    int min_w = 1;
    if (stack.empty()) min_w = (N + 1) / 2;

    int max_s = min(N - r + 1, N - c + 1);
    max_s = min(max_s, N - 1);

    bool placed = false;

    for (int w = max_s; w >= min_w; --w) {
        if (can_place(r, c, w)) {
            set_square(r, c, w, 1);
            stack.push_back({r, c, w});
            filled_area += w * w;

            start_row = r;
            placed = true;

            if (verbose) {
                cout << "    + Place " << w << "x" << w
                     << " at (" << r << ", " << c <<
")" << endl;
            }

            break;
        }
    }
    if (!placed) {
        backtrack = true;
        cout << "    Unable to place square in ("
              << r << ", " << c << ")" << endl;
    }
}

}

if (backtrack) {
    if (stack.empty()) { // Следующий limit
        cout << "    Backtrack. The stack is empty, moving
on to the next limit" << endl;
        break;
    }

    bool found_alternative = false;

    while (!stack.empty()) {
        Move last = stack.back();
        stack.pop_back();

        set_square(last.r, last.c, last.w, 0);
        filled_area -= last.w * last.w;

        start_row = last.r;

```

```

        if (verbose) {
            cout << "    - Remove " << last.w << "x" <<
last.w
            << " form (" << last.r << ", " << last.c
<< ")" << endl;
        }

        int next_w = last.w - 1;
        int min_w_for_level = (stack.empty()) ? ((N + 1) /
2) : 1;

        bool placed_smaller = false;
        for (int w = next_w; w >= min_w_for_level; --w) {
            if (can_place(last.r, last.c, w)) {
                set_square(last.r, last.c, w, 1);
                stack.push_back({last.r, last.c, w});
                filled_area += w * w;
                placed_smaller = true;

                if (verbose) {
                    cout << "    Reduce the size to " << w
<< "x" << w << endl;
                    cout << "    Placing a square " << w <<
"x" << w
                    << " in position (" << last.r <<
", " << last.c << ")" << endl;
                }

                break;
            }
        }

        if (placed_smaller) {
            found_alternative = true;
            break;
        }
    }

    if (!found_alternative && stack.empty()) {
        cout << "    Backtrack. All options exhausted for
limit = " << limit << endl;
        break;
    }
}

}

return 0;
}

```